

```

    if (New_red_count == 10)
    {
        Red_input=Current_red;
        New_red_found = 1;
    }
}
else
    New_red_count=0;

if (Current_green != Green_input)
{
    if (New_green_count<10)
    {
        New_green_count++;
        if (New_green_count == 10)
        {
            Green_input=Current_green;
            New_green_found = 1;
        }
    }
}
else
    New_green_count=0;
}

void Set_time(void)
{
    //Program comes here at every 5 ms interrupt

    One_tick_over = 1;
    Ticks++;
    Refresh_grid();
    Sense_input();
    if (Ticks == 100)
    //500 ms
    {
        Half_sec_over = 1;
    }

    if (Ticks == 200)
    //1000 ms
    {
        Ticks = 0;
        Secs++;
        One_sec_over = 1;
    }

    if (Secs == 60)
    {
        Mins++;
        One_min_over = 1;
        Secs = 0;
    }
}

/*general purpose routines are here.do not
use any hardware specific functions*/

void Binary2segment(unsigned int val)
{
    Seg_hun = Numeric_table[val/100];
    val = val%100;
    Seg_ten = Numeric_table[val/10];
    Seg_unit = Numeric_table[val%10];
}

void setup()
{
    //On power on call this routine for set
    all pins directions for In/Out
    Set_pins_dir();

    //Initialize Timer
    init_timer();
    Act_setup();
    K=0x01;
    digitalWrite(RE_DE,LOW); // step
2 done
    Serial.begin(9600);
}

void loop()
{
    if (One_tick_over)
    {
        One_tick_over=0;
        Act_one_Tick_over();
    }

    if (Half_sec_over)
    {
        Half_sec_over=0;
        Act_half_sec_over();
    }

    unsigned char Mode, Learn_red, Verify_red,time4input_active,Phase[2],z=0,Input_color[2],j;
    unsigned int Remaining_time[2],Learn_time[2],Verify_time[2],Display_time[2];
    unsigned char Serial_timeout, Serial_count, Serial_data, Slave_phase, Master_display, Packet_count;
    unsigned char Master_learn_time;
    unsigned int Slave_time=0;
    unsigned char Count_learn_time,Wait4_inactive[2],Wait4_active[2],TEST_MODE,Display_on,SELF_TEST_MODE,MS_MODE;
    // flags
    #define MASTER_MODE 1
    #define SLAVE_MODE 0
    #define START_BYTE 85
    #define DISPLAY_BLANK 1000
    #define DP_BLINK 1001
    #define ERROR_CODE 1002
    #define POWER_ON 0
    #define LEARN 1
    #define VERIFY 2
    #define DISPLAY_REMAINING_TIME 3
    #define TOLERANCE 2
    void Reset_serial_packet(void)
    {
        Serial_count = 0;
    }

    void Act_one_tick_over ()
    {
        if (MS_MODE == MASTER_MODE)
        {
            if (SELF_TEST_MODE)
                return;

            if(Red_input == ACTIVE_INPUT) && (Green_input == ACTIVE_INPUT)
            {
                Time4input_active=0;
                Count_learn_time = 0 ;
                for(j=0;j<2;j++)
                {
                    Wait4_inactive[j]=1;
                    Wait4_active[j]=0;
                    Phase[j]=LEARN;
                }

                Input_color[0]=Red_input;
                Input_color[1]=Green_input;
                for(j=0;j<2;j++)
                {
                    if (Wait4_inactive[j]==1)
                    {
                        if (Input_color[j] == INACTIVE_IN-PUT)
                        {
                            Wait4_active[j]=1;
                            Wait4_inactive[j]=0;
                            Phase[j] = LEARN;
                        }
                    }
                    //if wait4red_inactive
                    for(j=0;j<2;j++)
                    {
                        if (Wait4_active[j]==1)
                        {
                            if (Input_color[j] == ACTIVE_INPUT)
                            {
                                z=j;
                                Wait4_active[j]=0;
                                if(j==0)
                                    Color=COLOR_RED;
                                else
                                    Color=COLOR_GREEN;
                                Time4input_active=0;
                                Count_learn_time=1;
                                if (Phase[j] == DISPLAY_REMAIN-
ING_TIME)
                                {
                                    Remaining_time[j]=Learn_time[j];
                                    Display_on=1;
                                }
                                //if wait4red_active
                                if (Display_on == 1)
                                {
                                    if (Input_color[z] == INACTIVE_INPUT)
                                    {
                                        Wait4_active[z]=1;
                                        Display_on=0;
                                        Blankout_display();
                                        Send_packet(DISPLAY_BLANK);
                                        if (Remaining_time[z]>TOLERANCE)
                                            Phase[z]=LEARN;
                                        Wait4_active[z]=1;
                                    }
                                }

                                if(Phase[z] ==DISPLAY_REMAINING_TIME )
                                {
                                    if(Input_color[z]==ACTIVE_INPUT)
                                    {
                                        if(Display_on==0)
                                        {
                                            if((Time4input_active-Learn_time[z])>TOLERANCE)
                                            {
                                                Phase[z]=LEARN;
                                            }
                                        }
                                    }

                                    if (Count_learn_time==1)
                                    {
                                        if (Input_color[z] == INACTIVE_INPUT)
                                        {
                                            Count_learn_time = 0 ;
                                            Blankout_display();
                                            Send_packet(DISPLAY_BLANK);
                                            Wait4_active[z]=1;
                                            if (Phase[z] == LEARN)
                                            {
                                                Learn_time[z]=Time4input_active;
                                                Phase[z] = VERIFY;
                                            }
                                            // learn phase
                                            else if (Phase[z] == VERIFY)
                                            {
                                                Blankout_display();
                                                Send_packet(DISPLAY_BLANK);
                                                Verify_time[z]=Time4input_active;
                                                if (Verify_time[z] > Learn_time[z])
                                                {
                                                    if (Verify_time[z]-Learn_time[z]<TOLERANCE)
                                                    {
                                                        Learn_time[z]=Verify_time[z];
                                                        Phase[z]=DISPLAY_REMAIN-
ING_TIME;
                                                    }
                                                    else
                                                    {
                                                        Phase[z]=VERIFY;
                                                        Learn_time[z]=Verify_time[z];
                                                    }
                                                    else if (Verify_time[z] < Learn_time[z])
                                                    {
                                                        if ((Learn_time[z]-Verify_time[z])<TOLERANCE)
                                                        {
                                                            Learn_time[z]=Verify_time[z];
                                                            Phase[z]=DISPLAY_REMAIN-
ING_TIME;
                                                        }
                                                        else
                                                        {
                                                            Phase[z]=VERIFY;
                                                            Learn_time[z]=Verify_time[z];
                                                        }
                                                    }
                                                    else
                                                    {
                                                        Phase[z] = DISPLAY_REMAIN-
ING_TIME;
                                                    }
                                                }
                                            }
                                        }
                                    }
                                }
                            }
                        }
                    }
                }
            }
        }
    }
}

```